Trekking Management Application - V2

Modern Application Development II, project report, May 2026 term

Name	Navon John Lukose
Roll number	24F2003934
E-mail	24f2003934@ds.study.iitm.ac.in
Repository	https://github.com/Navonjl/mad2-trekking-management (private)

Problem statement

Trekking groups run their trips on spreadsheets and phone calls, which makes trek approvals, registrations, slot availability and booking records hard to keep straight. The application gives three kinds of user one shared set of data with different permissions over it. An admin creates trek routes, adds staff, assigns them to treks, and oversees every user and booking. Trek staff run only the treks assigned to them, setting slots and status and seeing who has signed up. Trekkers register themselves, search what is on offer, book a place, and keep a history of where they have been.

Approach

The application is a JSON API with a separate single-page front end. The same data has to appear three different ways, so putting every rule in the API means the browser cannot skip a check, and the three dashboards become three sets of components reading one set of endpoints.

Four decisions account for most of the design.

The first is a single user table for all three roles. Admin, staff and trekker store the same things and differ only in what they may do, so they share a `user` table with roles in a many-to-many join. Login is then one code path instead of three, and a role check is a decorator on the view. The two fields only staff have, employee code and duty status, sit in a separate `staff_profile` table so they are not NULL for every trekker.

The second is keeping the booking rules as plain conditions. The four rules in the problem statement, that the trek must exist, be Open, have a free slot, and not already be booked by this user, are four numbered `if` statements in `book_trek()`.

The third is the one exception to that. Taking a slot is not a plain `if`. Reading the count, checking it, then subtracting one is three steps, so two people pressing Book on the last slot can both pass the check. Instead a single conditional `UPDATE` decrements the count only while a slot is free and reports how many rows it changed. Zero rows means the trek filled up first.

The fourth is storing `available_slots` rather than counting bookings. Counting would mean a `COUNT` per trek on every listing. The stored number changes in the same request that creates the booking, so the two cannot drift.

Frameworks and libraries

Only the frameworks named in the problem statement are used. Nothing else was added on either side.

Layer	Technology	Version
API	Flask	3.1.3
ORM	Flask-SQLAlchemy / SQLAlchemy	3.1.1 / 2.0.51
Database	SQLite	built in
Auth	Flask-Security-Too	5.8.1
Cache	Flask-Caching with Redis	2.4.1 / 6.4.0
Jobs	Celery with Redis	5.6.3
UI	Vue 3 with Vue Router	3.5.40 / 5.2.0
Styling	Bootstrap	5.3.8
Charts	Chart.js	4.5.1
Build	Vite	8.1.5

No CORS library is needed, because the Vite dev server proxies `/api` to Flask and every request is therefore same-origin. There is no `python-dotenv` either: the `.env` file is read by an eight-line function in `app/config.py` using only the standard library.

Redis does two jobs across three logical databases. Database 0 is the Celery broker, 1 is the response cache, 2 is the Celery result backend.

Database design

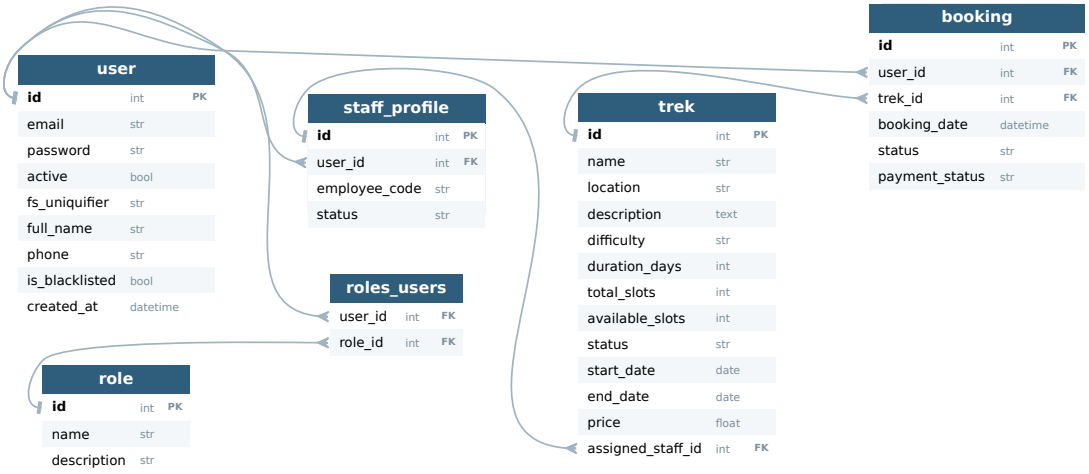


Figure 1: ER diagram

The diagram is generated from the SQLAlchemy models by graphviz, so it matches the schema exactly. Difficulty is Easy, Moderate or Hard. Trek status is Pending, Approved, Open, Closed or Completed. Booking status is Booked, Cancelled or Completed.

The `user` table also carries two-factor and WebAuthn columns supplied by Flask-Security's model mixin, which this project does not use. The schema is created by `db.create_all()` in the `flask init-db` command, not by hand.

API endpoints

Everything answers JSON. Every route except login and register needs an `Authentication-Token` header.

Authentication: `POST /api/accounts/login?include_auth_token`, `POST /api/accounts/logout`, `POST /api/register` (trekkers only), `GET /api/me`, `PUT /api/profile`.

Trekker: `GET /api/treks` (filters `q`, `difficulty`, `location`, `min_days`, `max_days`; cached in Redis), `POST /api/bookings`, `GET /api/bookings`, `POST /api/bookings/<id>/cancel`, `POST /api/exports/bookings`, `GET /api/exports/<task_id>`, `GET /api/exports/download/<filename>`.

Admin: `GET /api/admin/stats` (cached); `GET POST /api/admin/treks` and `PUT DELETE /api/admin/treks/<id>`; `PUT /api/admin/treks/<id>/assign-staff`; `GET POST /api/admin/staff`; `GET /api/admin/users`; `POST /api/admin/users/<id>/{deactivate,reactivate,blacklist,unblacklist}`; `GET /api/admin/bookings`; `POST /api/admin/bookings/<id>/mark-paid`.

Trek staff: `GET /api/staff/dashboard`, `PUT /api/staff/treks/<id>`, `GET /api/staff/treks/<id>/participants`. Each of these checks the trek is assigned to the caller and answers 403 otherwise.

Deactivating a user blocks login. Blacklisting lets them log in but refuses booking, cancelling and exporting.

Background jobs

Job	Trigger	What it does
`send_daily_reminders`	daily, 18:00	messages every trekker booked on a trek starting within three days, with the date and joining instructions. Posts to a Google Chat space through an incoming webhook, or e-mail if none is set
`send_monthly_activity_report`	1st, 06:00	HTML report of the previous month, covering treks conducted, users who took part and the five most popular treks, e-mailed to the admin
`export_bookings_csv`	user, from their dashboard	writes that trekker's history to CSV and alerts them when it is ready

The Celery instance is created in `app/celery_utils.py` and configured from the `CELERY` block in `app/config.py`. A small `FlaskTask` base class runs each task inside an application context, which is why the tasks use `db.session` directly.

Additional features

The trek listing is cached in Redis for 60 seconds and cleared whenever a trek changes. Caching is skipped for admin and staff, because the cache key does not include the role and an admin request would otherwise put unapproved treks into the listing a trekker sees. The admin dashboard draws a Chart.js bar chart of the most popular treks. The reminder job can post to Google Chat. An admin can record payment on a booking. Forms use HTML5 validation with matching checks in the API, and the Bootstrap layout works at phone width. A `flask seed-demo` command loads staff, treks, trekkers and bookings so the application can be shown straight away.

Video presentation

Drive link: [to be added]